

VisionInspectAI-

**Manufacturing Defect Detection & Quality
Inspection System**

Milestone-2 Report

Yash Goyal

Team-2

Module - AI

LIST OF CONTENTS

1. Work Summary.....	3
2. System Architecture.....	4
3. PatchCore Anomaly Detection.....	5-7
4. EfficientNet-B0 Defect Classification.....	8-10
5. Prediction Pipeline.....	11-12
6. Model Evaluation.....	13-16
7. Challenges.....	17
8. Conclusion.....	18

Chapter -1: Milestone 2 Work Summary

1.1 Modules Developed During Milestone 2

The following software modules were designed, implemented, integrated, and evaluated during Milestone 2 of the VisionInspect AI project.

S. No.	Module / Component	Work Completed
1	PatchCore Anomaly Detection	Developed and trained the PatchCore model for industrial anomaly detection using the MVTec AD dataset.
2	EfficientNet-B0 Defect Classifier	Developed and trained a universal EfficientNet-B0 model for multi-class defect classification.
3	Classifier Dataset Pipeline	Created a dataset builder for automatic label mapping, train-validation splitting, and DataLoader generation.
4	Model Training Pipeline	Implemented training workflows, checkpoint saving, and validation monitoring for both models.
5	Prediction Pipeline	Integrated PatchCore and EfficientNet-B0 into a unified inference pipeline for defect detection and classification.
6	Model Evaluation	Evaluated both models using Accuracy, Precision, Recall, F1 Score, ROC-AUC, Classification Report, and Confusion Matrix.

1.2 Tools and Technologies used

Category	Technology
Programming Language	Python
Deep Learning Framework	PyTorch
Anomaly Detection	PatchCore
Classification Model	EfficientNet-B0
Dataset	MVTec AD
Image Processing	OpenCV
Evaluation	Scikit-learn
Visualization	Matplotlib

CHAPTER 2: SYSTEM ARCHITECTURE

2.1 Overall System Architecture

The VisionInspect AI system consists of two primary AI modules: the PatchCore anomaly detection model and the EfficientNet-B0 defect classification model. The system first determines whether a product is normal or defective. If a defect is detected, the classifier identifies the specific defect type and displays the prediction along with the confidence score.

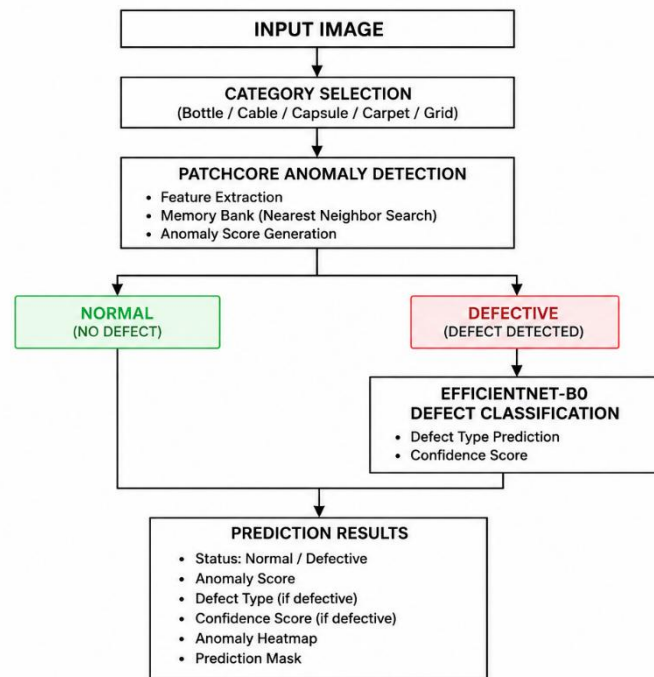


Figure 2.1 Overall System Architecture Diagram

2.2 System Workflow

The inspection workflow consists of the following steps:

1. User selects the product category.
2. Input image is loaded.
3. PatchCore performs anomaly detection.
4. If the image is normal, the inspection process terminates.
5. If defective, the image is forwarded to EfficientNet-B0.
6. EfficientNet-B0 predicts the defect type.
7. Final prediction and confidence score are displayed.

CHAPTER 3: PATCHCORE ANOMALY DETECTION

3.1 Overview

PatchCore is an anomaly detection model designed to identify defects by learning the characteristics of normal samples. Instead of classifying specific defect types, it detects whether an input image deviates from the learned normal patterns. In VisionInspect AI, PatchCore serves as the first stage of the inspection pipeline by determining whether a product is **Normal** or **Defective**.

Why PatchCore?

- Trained using only normal images.
- Detects unseen or unknown defects effectively.
- Does not require defect-specific annotations.
- Suitable for industrial quality inspection tasks.
- Achieves high anomaly detection performance on the MVTec AD dataset.

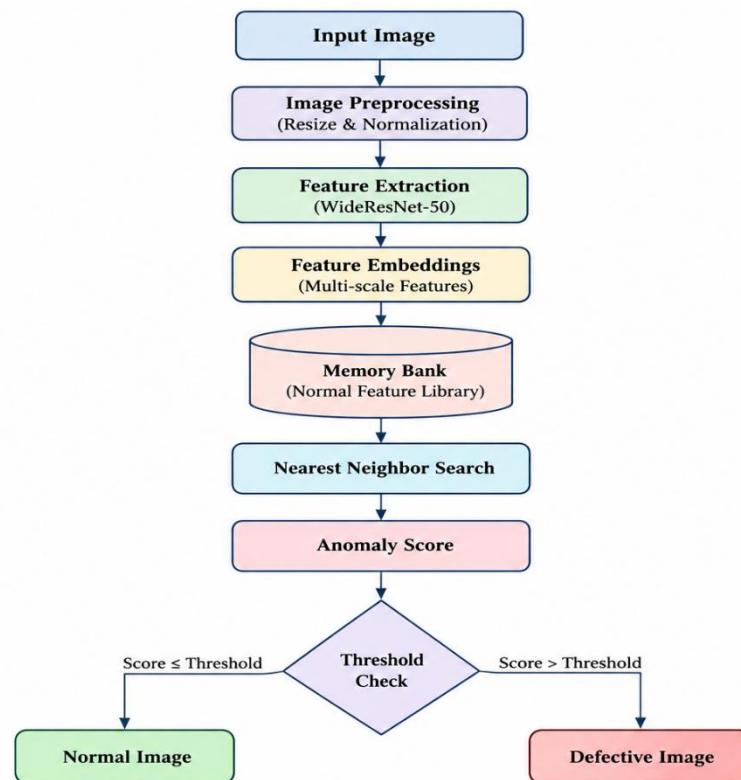


Figure 3.1 Internal Working of the PatchCore Anomaly Detection Model

3.2 Working Principle

The PatchCore model performs anomaly detection through the following stages:

1. **Input Image**
 - Receives the product image for inspection.
2. **Image Preprocessing**
 - Resizes and normalizes the image before feature extraction.
3. **Feature Extraction**
 - Extracts deep visual features using a pretrained WideResNet-50 backbone.
4. **Feature Embedding**
 - Generates multi-scale feature representations from the extracted features.
5. **Memory Bank Construction**
 - Stores representative feature embeddings of normal training images.
6. **Nearest Neighbor Search**
 - Compares the input image features with the stored normal feature embeddings.
7. **Anomaly Score Computation**
 - Calculates an anomaly score based on feature similarity.
8. **Decision**
 - If the anomaly score exceeds the predefined threshold, the image is classified as **Defective**; otherwise, it is classified as **Normal**.

3.3 Model Configuration

Parameter	Value
Model	PatchCore
Framework	Anomalib
Backbone	Wide ResNet50-2
Feature Layers	Layer2, Layer3
Coreset Sampling Ratio	0.1
Number of Nearest Neighbors	9
Pretrained Backbone	Yes
Input Image Size	256 × 256
Batch Size (Training)	32
Batch Size (Evaluation)	32

3.4 Training Process

The PatchCore model was trained using the following procedure:

1. Load the normal training images.
2. Apply image preprocessing.
3. Extract deep feature embeddings.
4. Construct the memory bank using representative normal features.
5. Save the trained PatchCore model for inference.

```
Selecting Coreset Indices.: 100%|#####| 27021/27032 [28:13<00:00, 16.35it/s]
Selecting Coreset Indices.: 100%|#####| 27023/27032 [28:13<00:00, 16.43it/s]
Selecting Coreset Indices.: 100%|#####| 27025/27032 [28:13<00:00, 16.43it/s]
Selecting Coreset Indices.: 100%|#####| 27027/27032 [28:13<00:00, 16.46it/s]
Selecting Coreset Indices.: 100%|#####| 27029/27032 [28:13<00:00, 16.45it/s]
Selecting Coreset Indices.: 100%|#####| 27031/27032 [28:13<00:00, 16.48it/s]
Selecting Coreset Indices.: 100%|#####| 27032/27032 [28:13<00:00, 15.96it/s]

Epoch 0/0 5/5 0:00:56 • 0:00:00 0.11it/s `Trainer.fit` stopped: `max_epochs=1` reached.
Epoch 0/0 5/5 0:00:56 • 0:00:00 0.11it/s
grid training completed.

=====
Training Category : hazelnut
=====
█

Selecting Coreset Indices.: 100%|#####| 40032/40037 [1:11:05<00:00, 10.62it/s]
Selecting Coreset Indices.: 100%|#####| 40034/40037 [1:11:05<00:00, 10.63it/s]
Selecting Coreset Indices.: 100%|#####| 40036/40037 [1:11:05<00:00, 10.66it/s]
Selecting Coreset Indices.: 100%|#####| 40037/40037 [1:11:05<00:00, 9.39it/s]

Epoch 0/0 7/7 0:01:24 • 0:00:00 0.11it/s `Trainer.fit` stopped: `max_epochs=1` reached.
Epoch 0/0 7/7 0:01:24 • 0:00:00 0.11it/s
hazelnut training completed.
```

Figure 3.2: PatchCore Training

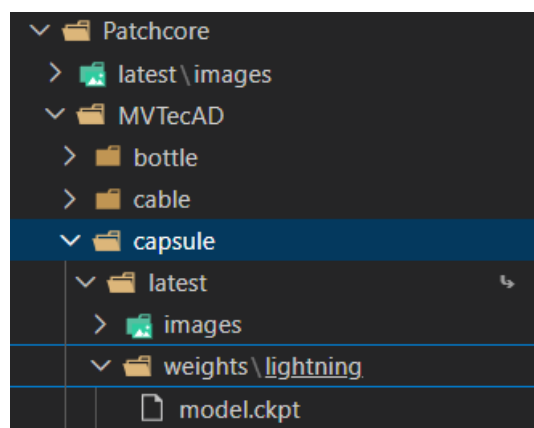


Figure 3.3: Saved Model Checkpoint

CHAPTER 4: EFFICIENTNET-B0 DEFECT CLASSIFICATION

4.1 Overview

EfficientNet-B0 is a convolutional neural network (CNN) designed for image classification. In VisionInspect AI, it is used as the second stage of the inspection pipeline to identify the specific defect type after PatchCore detects an anomaly. The model classifies defective images into their respective defect categories and provides a confidence score for the prediction.

Why EfficientNet-B0?

- Lightweight and computationally efficient.
- High classification accuracy with fewer parameters.
- Well-suited for industrial image classification.
- Fast inference for real-time inspection.
- Easy to fine-tune on custom defect datasets.

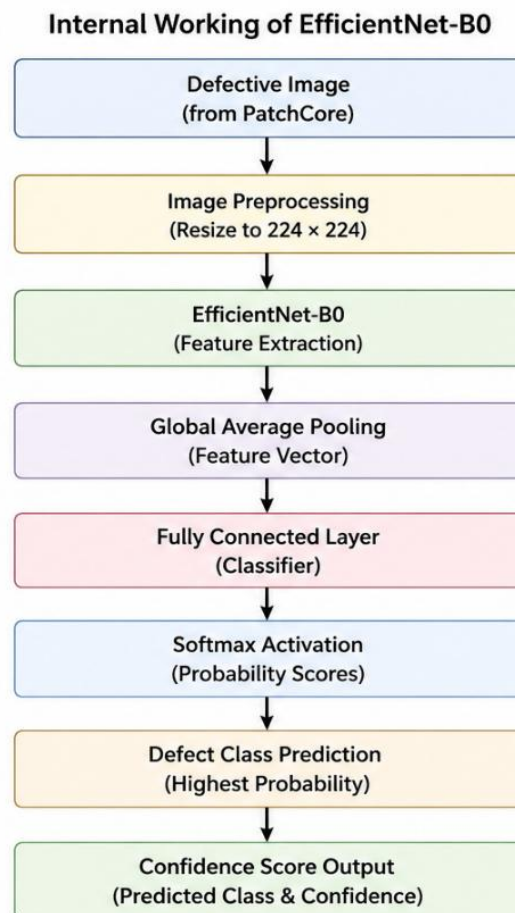


Figure 4.1 Internal Working of the EfficientNet-B0 Classification Model

4.2 Working Principle

The EfficientNet-B0 classifier performs defect classification through the following stages:

1. **Input Image**
 - Receives the defective image from the PatchCore module.
2. **Image Preprocessing**
 - Resizes and normalizes the image to the required input dimensions.
3. **Feature Extraction**
 - Extracts hierarchical visual features using EfficientNet-B0.
4. **Global Average Pooling**
 - Converts feature maps into a compact feature vector.
5. **Fully Connected Layer**
 - Maps extracted features to the defect classes.
6. **Softmax Layer**
 - Computes the probability for each defect category.
7. **Prediction**
 - Outputs the defect class with the highest confidence score.

4.3 Dataset Preparation

Briefly describe the classifier dataset:

- Defective images from selected MVTec AD categories were used.
- Images were organized into class-wise folders.
- Automatic label mapping was generated.
- Training and validation datasets were created using DataLoaders.

4.4 Model Configuration

Parameter	Value
Model	EfficientNet-B0
Input Size	224 × 224
Training Dataset	Classifier Dataset

Output Classes	Defect Categories
Framework	PyTorch
Loss Function	CrossEntropyLoss
Optimizer	Adam
Learning Rate	1e-4
Batch Size	32
Epochs	20

4.5 Training Process

1. Load the classifier dataset.
2. Apply image preprocessing.
3. Train EfficientNet-B0 using labeled defect images.
4. Validate the model after each epoch.
5. Save the best-performing model checkpoint.

```

OUTPUT  DEBUG CONSOLE  PROBLEMS  PORTS  QUERY RESULTS  TERMINAL
INFO | zipper_fabric_interior | 16
INFO | zipper_rough | 17
INFO | zipper_split_teeth | 18
INFO | zipper_squeezed_teeth | 16
INFO | =====
Downloading: "https://download.pytorch.org/models/efficientnet_b0_rn188-53529a17.pth" to C:\Users\Yash Goyal/.cache\torch\hub\checkpoints\efficientnet_b0_rn188-53529a17.pth
100%|#####| 20.5M/20.5M [00:05<00:00, 3.98MB/s]
INFO | =====
INFO | Epoch 1/20 | 0/32 [00:00<?, ?it/s]C
:\Projects\VisionInspect AI\venv\lib\site-packages\torch\utils\data\data_loader.py:1095: UserWarning: 'pin_memory' argument is set as true but no acc
elerator is found, then device pinned memory won't be used.
  success()
  
```

Figure 4.2: EfficientNet-B0 Training

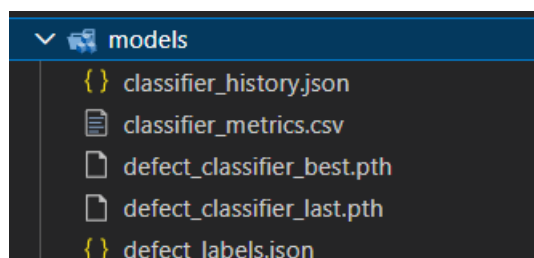


Figure 4.3 Saved best-performing model checkpoint

CHAPTER 5: PREDICTION PIPELINE

5.1 Overview

The prediction pipeline integrates the PatchCore anomaly detection model with the EfficientNet-B0 classification model to perform automated product inspection. PatchCore first determines whether the input image is normal or defective. If a defect is detected, the image is forwarded to EfficientNet-B0 for defect classification. The final output displays the inspection status, predicted defect class, and confidence score.

5.2 Prediction Workflow

Use point-wise steps:

1. Load the trained PatchCore model.
2. Load the trained EfficientNet-B0 classifier.
3. Read the input image.
4. User selects the product category.
5. PatchCore performs anomaly detection.
6. If the image is **Normal**, display the result and terminate.
7. If the image is **Defective**, pass the image to EfficientNet-B0.
8. EfficientNet-B0 predicts the defect class.
9. Display the final prediction with the confidence score.

Note: In the current implementation, the product category is selected manually by the user. In future versions, an automatic category detection module can be integrated to identify the product category before the anomaly detection stage, resulting in a fully automated inspection workflow.

5.3 Prediction Outputs

```
VISIONINSPECT AI

Image:-
Image Name      : 004.png

PatchCore:-
Category        : pill
Status          : Normal
Anomaly Score    : 0.2909
```



Figure 5.1: Normal prediction Outputs

```

OUTPUT  DEBUG CONSOLE  PROBLEMS  PORTS  QUERY RESULTS  TERMINAL
VISIONINSPECT AI

Image:-
Image Name      : 003.png

PatchCore:-
Category        : pill
Status          : Defective
Anomaly Score   : 1.0

Classifier:-
Category        : pill
Defect          : crack
Confidence      : 80.12%
  
```

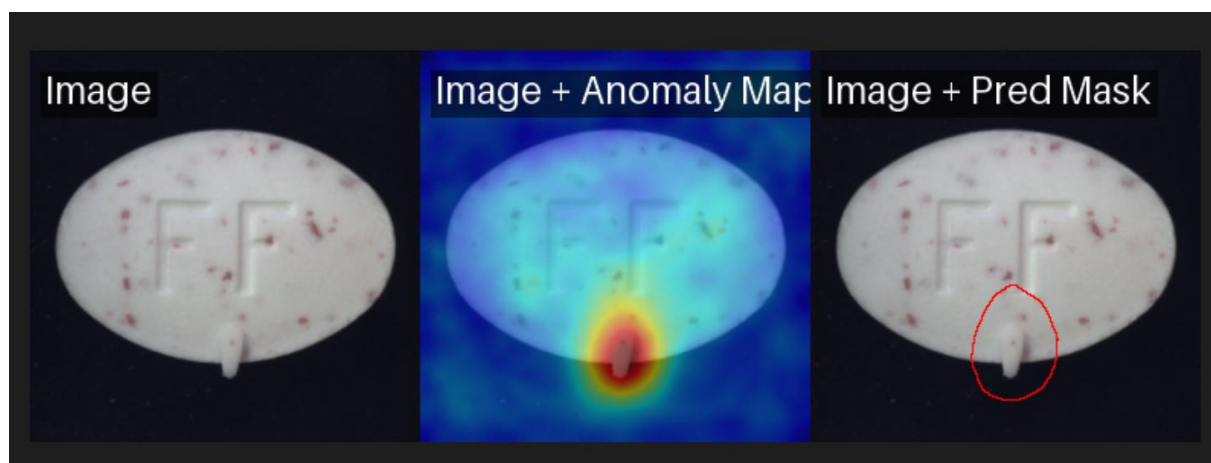


Figure 5.2: Defective prediction Outputs

CHAPTER 6: MODEL EVALUATION

6.1 Evaluation Overview

The trained PatchCore and EfficientNet-B0 models were evaluated using standard performance metrics to measure their effectiveness. PatchCore was evaluated for anomaly detection performance, while EfficientNet-B0 was evaluated for defect classification accuracy. The evaluation provides a quantitative assessment of the proposed inspection system.

6.2 Evaluation Metrics

Metric	Description
Accuracy	Percentage of correctly predicted samples.
Precision	Measures the correctness of positive predictions.
Recall	Measures the ability to detect actual positive samples.
F1-Score	Harmonic mean of Precision and Recall.
ROC-AUC	Measures the model's ability to distinguish between classes.
Confusion Matrix	Summarizes prediction performance for each class.
Classification Report	Provides Precision, Recall, F1-Score, and Support for each class.

6.3 PatchCore Evaluation

Overall Performance

Metric	Value
Accuracy	95.84%
Precision	97.82%
Recall	96.34%
F1-Score	97.05%
ROC-AUC	98.19%

Category	Total Images	Normal Images	Defective Images	Accuracy	Precision	Recall	F1 Score	ROC AUC
bottle	83	20	63	0.9880	1.0000	0.9841	0.9920	1.0000
cable	150	58	92	0.9600	0.9674	0.9674	0.9674	0.9843
capsule	132	23	109	0.9621	0.9906	0.9633	0.9767	0.9900
carpet	117	28	89	0.9573	1.0000	0.9438	0.9711	0.9876
grid	78	21	57	0.9487	0.9818	0.9474	0.9643	0.9866
hazelnut	110	40	70	0.9909	1.0000	0.9857	0.9928	1.0000
leather	124	32	92	0.9919	1.0000	0.9891	0.9945	1.0000
metal_nut	115	22	93	0.9826	1.0000	0.9785	0.9891	0.9990
pill	167	26	141	0.9222	0.9776	0.9291	0.9527	0.9438
screw	160	41	119	0.9125	0.9412	0.9412	0.9412	0.9647
tile	117	33	84	0.9915	1.0000	0.9881	0.9940	1.0000
toothbrush	42	12	30	0.9048	0.9062	0.9667	0.9355	0.9194
transistor	100	60	40	0.9600	0.9500	0.9500	0.9500	0.9912
wood	79	19	60	0.9367	0.9825	0.9333	0.9573	0.9833
zipper	151	32	119	0.9669	0.9750	0.9832	0.9791	0.9790

Average Metrics	
Total Images	: 115.0000
Normal Images	: 31.1333
Defective Images	: 83.8667
Accuracy	: 0.9584
Precision	: 0.9782
Recall	: 0.9634
F1 Score	: 0.9705
ROC AUC	: 0.9819

Figure 6.1: PatchCore Evaluation Results

Observation

- High anomaly detection accuracy.
- Excellent precision and recall.
- Strong ability to distinguish normal and defective products.

6.4 EfficientNet-B0 Evaluation

Overall Performance

Metric	Value
Accuracy	80.16%
Precision	82.01%
Recall	80.16%
F1-Score	78.90%

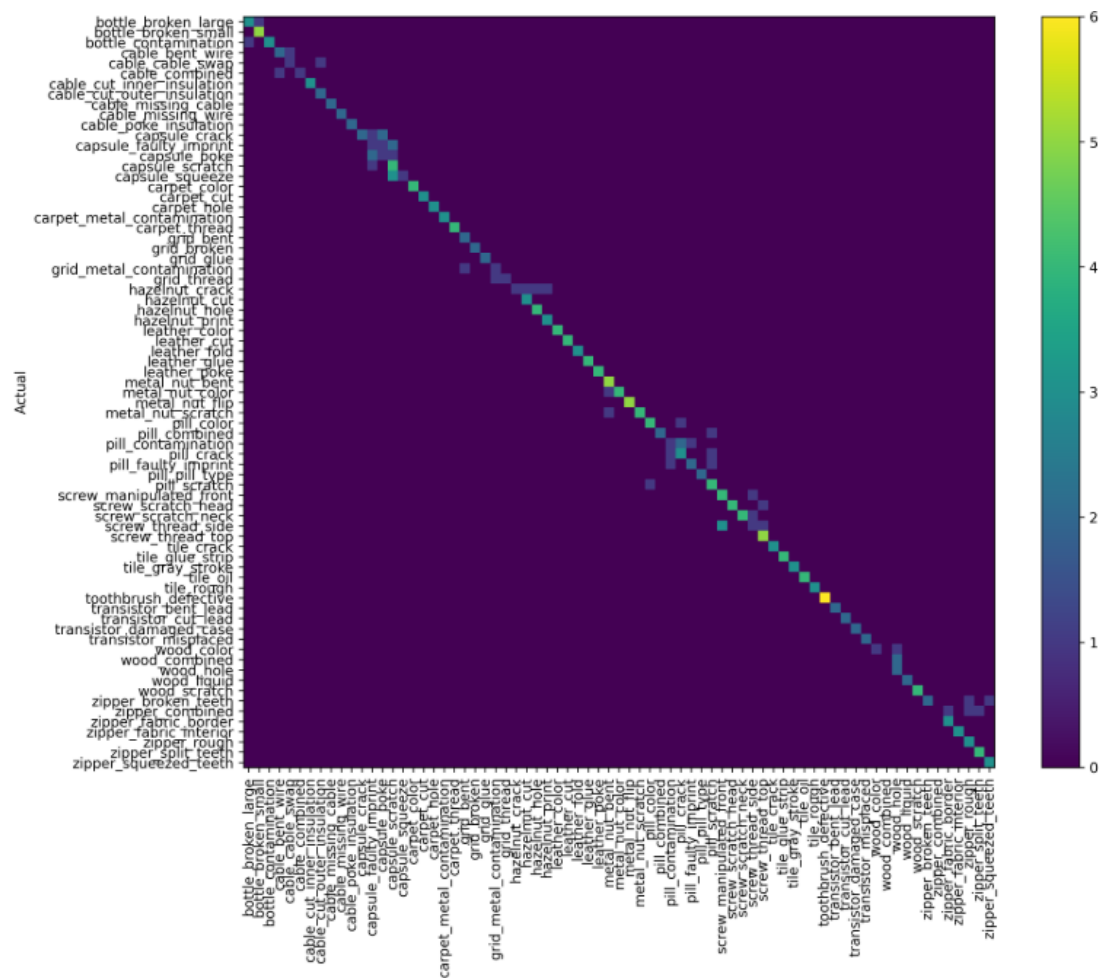


Figure 6.2: Confusion Matrix (EfficientNet-B0)

AI > outputs > evaluation > classifier_metrics.txt	
1	Accuracy : 0.8016
2	Precision : 0.8201
3	Recall : 0.8016
4	F1 Score : 0.7890
5	

Figure 6.3: Classification Report

Observation

- Good classification accuracy.
- Effective classification across multiple defect categories.
- Some confusion observed between visually similar defect classes.

6.5 Performance Comparison

Model	Task	Accuracy
PatchCore	Anomaly Detection	95.84%
EfficientNet-B0	Defect Classification	80.16%

6.6 Discussion

- PatchCore achieved excellent anomaly detection performance.
- EfficientNet-B0 accurately classified most defect categories.
- The two-stage pipeline reduces unnecessary classification of normal products.
- The combined system is suitable for industrial quality inspection tasks.

CHAPTER 7: CHALLENGES FACED

Challenge	Description	Solution Implemented
Large Dataset	The MVTec AD dataset contains multiple product categories and numerous high-resolution images.	The dataset was preprocessed and organized category-wise to simplify training and evaluation.
Model Training Time	Training deep learning models on CPU required significant processing time. It takes usually 25 mins to 1 hour to train each category depending upon images.	Training parameters were optimized to improve execution efficiency but not so much improved.
Dataset Preparation	Building a unified dataset for defect classification required consistent label mapping.	An automated dataset preparation pipeline was developed.
Model Integration	Combining anomaly detection and defect classification into a single workflow required careful pipeline design.	A two-stage prediction pipeline was implemented using PatchCore and EfficientNet-B0.
Performance Evaluation	Evaluating two different models required separate metrics and analysis.	Standard evaluation metrics and visualization techniques were used to assess performance.

CHAPTER 8: CONCLUSION

8.1 Conclusion

VisionInspect AI was successfully developed as an intelligent manufacturing defect detection and quality inspection system. The proposed two-stage architecture combines PatchCore for anomaly detection and EfficientNet-B0 for defect classification, enabling accurate identification of defective products and their corresponding defect categories. Experimental results demonstrate strong anomaly detection performance and effective defect classification, making the system suitable for industrial quality inspection applications. The modular architecture also provides flexibility for future enhancements and deployment in real-world manufacturing environments.

8.2 Key Achievements

- Developed an automated manufacturing defect inspection system.
- Implemented PatchCore for industrial anomaly detection.
- Developed an EfficientNet-B0-based defect classification model.
- Integrated both models into a unified prediction pipeline.
- Achieved 95.84% anomaly detection accuracy.
- Achieved 80.16% defect classification accuracy.
- Evaluated the system using standard performance metrics.